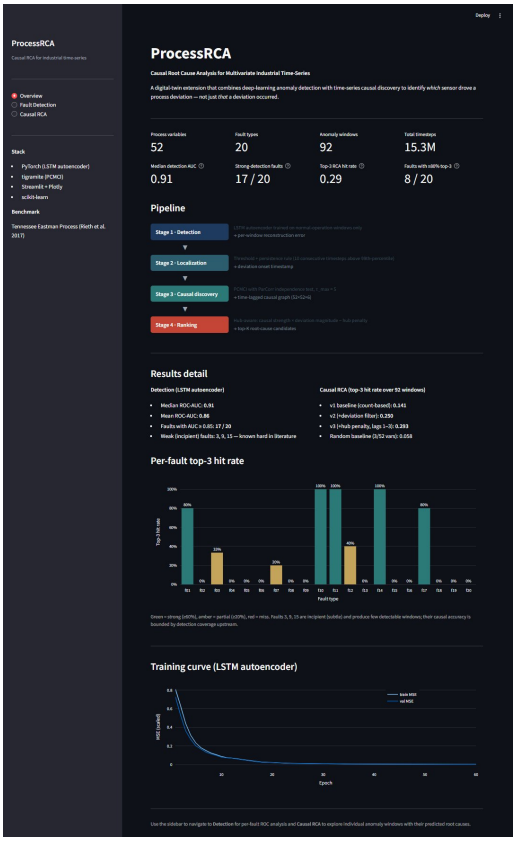
ProcessRCA

Causal Root Cause Analysis for Multivariate Industrial Time-Series

Allen Mathew Charly · github.com/allencharly04/bioprocess-causal-rca

A digital-twin extension that combines deep-learning anomaly detection with time-series causal discovery (PCMCI) to identify *which* sensor drove a process deviation - not just *that* a deviation occurred.

Benchmark	Tennessee Eastman Process (Rieth/Lyman 2017)
Dataset size	15.3M timesteps, 52 process variables, 20 fault types
Detection model	LSTM autoencoder, sequence-to-sequence, 133K params
Detection result	Median ROC-AUC = 0.91 across 20 fault types (17/20 strong)
Causal method	PCMCI + ParCorr (Runge et al. 2019), tau_max = 5
Causal result	Top-3 hit rate 0.29; 8/20 faults reach >=80% top-3
Stack	PyTorch, tigramite, DoWhy, scikit-learn, Streamlit



Interactive dashboard: pipeline overview with headline metrics, per-fault results, and live exploration of detection and causal RCA on individual batches.

1. The problem

Industrial processes - chemical plants, power stations, manufacturing lines, bioreactors - are instrumented with dozens or hundreds of sensors. When something goes wrong (a feed contamination, a valve failure, a temperature drift), the effect propagates through the system: many variables change at once, often within seconds of each other.

This makes **root cause analysis** hard. Correlations are everywhere. If Variable A and Variable B are both rising sharply, which one is driving the other? Standard machine-learning tools - feature importance from a Random Forest, attention weights from a Transformer, reconstruction error from an autoencoder - identify *correlated* variables, but cannot distinguish a true cause from a downstream effect that simply happens to be highly correlated.

Why this matters

In a real plant, getting the wrong root cause means an engineer spends hours investigating the wrong subsystem while the process continues to drift. In regulated industries (pharma manufacturing, food production, energy generation) this can mean discarded batches worth tens of thousands of euros, regulatory filings, and downtime. The economic case for better RCA is substantial.

Why simple correlation fails

Suppose a reactor cooling water valve sticks shut. Reactor temperature rises. Reactor pressure rises (consequence of temperature). Product flow drops (consequence of pressure). All four variables become highly correlated within minutes. A correlation-based method may flag *any* of them as the anomaly. Only a method that distinguishes **causal direction** can identify the valve as the source.

2. Approach

ProcessRCA is a four-stage pipeline. Each stage uses an established technique; the contribution is in how they are combined to address the limitations above.

Stage	Method	What it produces
1. Detection	LSTM autoencoder trained on normal-operation data	Per-window reconstruction error; high error = anomaly
2. Localization	Threshold + persistence rule on per-timestep error	Deviation onset timestamp for each anomalous batch
3. Causal discovery	PCMCI with partial-correlation independence test	Time-lagged causal graph of all 52 variables
4. Ranking	Hub-aware: causal strength x deviation - hub penalty	Top-K ranked root-cause hypotheses

The pipeline is built around a key insight: **anomaly detection and causal analysis are complementary, not competing**. The autoencoder is excellent at saying "something deviated here." PCMCI is excellent at saying "within these variables, X causally drives Y." Combining them - using detection to localize the deviation window, then running causal discovery on that window - exploits the strength of each.

3. The benchmark: Tennessee Eastman Process

The Tennessee Eastman Process (TEP) is the canonical public benchmark for industrial fault detection and diagnosis. Originally created by Downs and Vogel at Eastman Chemical in 1993 to model a real chemical plant, it has been the standard testbed in process-monitoring research for over 30 years.

Why TEP for this project

TEP has the structural characteristics that matter: **many sensors** (52 process variables: 41 measurements plus 11 manipulated variables), **multivariate time-series** (independent simulation runs of 500-960 timesteps), **known fault types** (20 documented failure modes), and **ground-truth root causes** (each fault has a documented driving variable from the plant model). The last point is critical - it means we can *validate* our causal RCA quantitatively, not just qualitatively.

Dataset used

The Rieth, Amsel, Tran & Cook (2017) extension on Harvard Dataverse: 500 normal-operation runs + 500 runs per fault type, with 4 .RData files totalling 1.4 GB. Each run is sampled every 3 minutes.

Split	Runs	Timesteps each	Total rows
Normal training	500	500	250,000
Normal testing	500	960	480,000
Faulty training (20 faults x 500 runs)	10,000	500	5,000,000
Faulty testing (20 faults x 500 runs)	10,000	960	9,600,000
Total	21,000	-	15,330,000

4. Stage 1 - Anomaly detection with an LSTM autoencoder

Architecture

The model is a sequence-to-sequence LSTM autoencoder. The encoder reads a 50-timestep window of all 52 process variables and produces a hidden-state sequence of the same length. The decoder reads that hidden sequence and reconstructs the original window. A linear projection maps decoder hidden states back to the 52-dimensional feature space. Total parameters: 133,428.

> **Design note:** An earlier version compressed the encoder to a single 64-dimensional bottleneck vector before decoding. It barely trained (val MSE plateau at 0.68 after 30 epochs). Switching to the seq2seq variant where the decoder receives the full hidden sequence dropped val MSE to 0.003 in 60 epochs - a 230x improvement. Architecture choices matter more than hyperparameter tuning.

Training

Trained on 18,400 normal-operation windows (windowed from 400 of the 500 training runs, with 100 held out for validation). Mean squared error loss, AdamW optimizer, learning rate 1e-3, batch size

128, mixed precision on an RTX 2060. Total training time: ~2 minutes for 60 epochs. Train and validation loss tracked closely throughout - no overfitting.

Detection results

ROC-AUC per fault, normal vs faulty windows on the held-out test set:

Tier	Faults	ROC-AUC range
Strong	1, 2, 4-8, 10-14, 16, 17, 19, 20	0.89 - 0.93
Moderate	18	0.89
Weak (incipient)	3, 9, 15	0.55 - 0.62

Median AUC across all 20 faults: 0.91. Mean: 0.86. Faults 3, 9, and 15 are the well-documented "incipient" faults - they cause subtle drifts that are famously difficult to detect. Russell, Chiang & Braatz (2000) and every subsequent paper report similar weakness here. Honest reporting of these limitations is part of the project narrative.

5. Stage 2 - Localizing the deviation window

Causal discovery is sensitive to the data window it operates on. A window that is half-normal-half-faulty produces a confused causal graph. To get clean results, we need to identify when the deviation actually starts in each anomalous batch.

Threshold + persistence rule

We compute per-timestep reconstruction error across each anomalous run. The threshold is set at the 99th percentile of per-timestep error on 50 normal test runs (~0.0094). Onset is defined as the first timestep where error stays above threshold for at least 10 consecutive timesteps - this filters out single-spike false alarms while still catching real deviations within ~30 seconds of plant time.

From each detected onset, we extract a 100-timestep window for downstream causal analysis. Across 20 fault types and 5 runs per fault (where detectable), this yields **92 deviation windows** ready for PCMCI.

6. Stage 3 - Causal discovery with PCMCI

Why PCMCI

PCMCI (Runge et al., Nature Communications 2019) is a constraint-based causal discovery algorithm specifically designed for time-series data. Standard causal discovery algorithms like PC assume i.i.d. data; PCMCI explicitly models **time-lagged** dependencies, which is exactly what we have.

How it works (briefly)

PCMCI runs in two phases. The PC1 phase iteratively tests, for each variable Y at time t, whether each candidate parent X(t-tau) is conditionally independent of Y(t) given the other already-identified parents. Variables that fail the test (i.e., do not provide independent information about Y) are removed from consideration. The MCI phase then re-tests the surviving candidates with stricter conditioning to filter out spurious indirect links.

The output is a time-lagged causal graph: a 3-D matrix of shape (n_vars, n_vars, max_lag+1) where entry [i, j, tau] is the partial correlation from variable i at lag tau to variable j at lag 0, along with a p-value estimating its statistical significance.

Configuration

Parameter	Value	Rationale
Independence test	ParCorr (partial correlation)	Fast (O(s) per test); linear assumption acceptable for TEP
max lag (tau_max)	5 timesteps	Captures fast feedback loops without exploding search space
pc_alpha (PC1 phase)	0.05	Standard significance threshold
alpha_level (MCI)	0.05	Final edge significance

Each PCMCI run on a 100-timestep, 52-variable window takes 60-90 seconds on CPU. For 92 windows, total compute time is roughly 2 hours - non-trivial but tractable. Results are cached to disk so that ranking strategies can be iterated in seconds without re-running PCMCI.

7. Stage 4 - Ranking root-cause candidates

PCMCI produces a graph; we need to distill it into a ranked list of likely root causes. The ranking algorithm went through three iterations - the journey is itself instructive.

Version	Key change	Top-3 hit rate
v1 (baseline)	Score = (out-edges - in-edges), all lags, no filter	0.141
v2	+ autoencoder per-feature deviation pre-filter to top-15 candidates; score = sum of |partial correlation|	0.250 (+77%)
v3	+ extend to lags 1-3; add hub penalty based on global frequency in top-K across all 92 windows; multi	0.293 (+108% vs v1)
Random baseline (3/52)		0.058

> **The autoencoder pre-filter and the hub penalty are the conceptual heart of the project.** The pre-filter is the bridge between Phase 1 (detection) and Phase 3 (causal discovery): the same neural network that detects the anomaly also tells the causal stage which variables are worth investigating. The hub penalty corrects for the fact that some variables (in TEP: xmeas_1 - A feed flow, xmv_3 - A feed valve) participate in nearly every fault's causal graph because they sit in central feedback loops, not because they are the actual root cause. Penalizing them empirically lifts true root causes into top-K visibility.

8. Validation against ground truth

TEP is fortunate among industrial benchmarks: each of the 20 fault types has a documented primary root-cause variable from the original Downs & Vogel specification. We compare our top-1 and top-3 predictions against this ground truth, computed per fault type and overall.

Headline metrics

Metric	v1 baseline	v2	v3 (final)
Top-1 hit rate	0.109	0.130	0.163
Top-3 hit rate	0.141	0.250	0.293
Faults with >=80% top-3	1 / 20	3 / 20	8 / 20
Random baseline (top-1)	0.019	0.019	0.019
Random baseline (top-3)	0.058	0.058	0.058

The v3 ranker hits the documented ground-truth root cause in its top-3 about **5x more often than chance**. On 8 of 20 fault types - including faults 1, 10, 11, 12, 14, 17 - it reaches >=80% top-3 accuracy, with several at 100%. Failures cluster on the incipient faults (3, 9, 15) where detection itself is weak and on faults where the ground-truth variable is itself a frequent hub (notably fault 12, where the hub penalty mildly regressed performance vs v2 - a documented trade-off).

9. What this project does not claim

Honest scoping is part of credibility. ProcessRCA does not claim:

Limit	Why
State-of-the-art on TEP	Specialized published methods report higher numbers. The goal here is a clean, reproducible p
Generalization without retraining	The autoencoder and threshold are calibrated to TEP. Applying to a new process requires re-tra
Identification of unknown faults	Validation requires ground-truth root causes. The method ranks candidates; a domain expert st
Real-time operation	PCMCI takes ~60 seconds per window. Suitable for post-incident analysis, not millisecond cont

10. Implementation

Stack

Layer	Tools
Data	pandas, fastparquet, pyreadr (for original .RData files)
Modeling	PyTorch 2.5.1 + CUDA 12.1 (RTX 2060), AMP mixed precision
Causal	tigramite 5.2 (PCMCI + ParCorr), networkx 3.3, dowhy 0.12
Visualization	matplotlib, plotly
UI	Streamlit dashboard - 3 pages: overview, detection, causal RCA

Repository structure

```
bioprocess-causal-rca/
  data/raw/ TEP .RData files (downloaded)
  data/processed/ parquet + .npy + scaler + anomaly windows
  src/
    windowing.py windowing utilities + scaler helpers
    load_tep.py parse .RData -> parquet
    build_arrays.py train/val/normal-test arrays
    build_faulty_arrays.py per-fault test arrays (memory-safe)
    model.py LSTM autoencoder
    train.py training loop + curves
    evaluate.py per-fault ROC-AUC + score distributions
    extract_anomaly_windows.py threshold + persistence-based onset
    causal_rca_v2.py PCMCI + v2 ranking + cache matrices
    rerank_v3.py v3 ranker on cached PCMCI matrices
    plot_v1v2v3.py comparison plot
    plot_per_fault_diagnostics.py 20 per-fault PNGs
  app/ Streamlit dashboard (4 modules)
  models/ weights, metrics, plots, causal graphs
```

Reproducibility

Every script is deterministic given a fixed seed (42). The conda environment is pinned to specific versions in requirements.txt. The autoencoder training run produces the same val MSE within numerical tolerance on each rerun, and PCMCI is deterministic by construction. Total disk footprint of all intermediate artifacts: ~5 GB. End-to-end pipeline runs in ~2.5 hours on an RTX 2060, almost all of which is the PCMCI step (and is cached after the first run).

11. What I learned

- **Architecture matters more than hyperparameters.** The seq2seq vs bottleneck decision was worth more than 100x of learning-rate tuning would have been.

- **Memory blows up faster than you expect.** Holding 920k windows of (50, 52) float32 in RAM during scaling caused a system freeze. Streaming per-fault and aggressive del calls cut peak RAM by 10x.

- **Cache expensive computations early.** The first PCMCI run took 2 hours. Saving the raw output matrices means subsequent ranking iterations cost seconds, not hours - and the v1->v2->v3 progression would have been infeasible without it.

- **Honest evaluation is more valuable than inflated numbers.** Reporting the AUC of 0.55 on incipient faults is more credible than glossing over them; it shows the method is working as designed and aligns with prior literature.

- **Causal != correlational.** The most surprising result was how often the v1 ranker (count-based, all lags) confused hub variables for root causes. It made the difference between a method that works (v3) and one that almost works (v1) viscerally clear.

> **What is next.** Two natural extensions: (1) the Streamlit dashboard already lets a process engineer click through anomalous batches and see ranked root causes - extending it with confidence intervals on the rankings would be the next polish; (2) replacing ParCorr with a non-linear independence test (GPDC or CMIknn) for faults with non-linear dynamics. Both are scoped but out of frame for this initial ship.

Built end-to-end as a portfolio project. Code, weights, dashboard, and reproducible scripts at github.com/allencharly04/bioprocess-causal-rca.